

regmap 深入理解

struct regmap 的本质、用法与内存模型

从"图纸 vs 机器"到句柄套路、函数参数、内核内存与工厂

配套仓库:github.com/fyy0619-cell/xiangmu · 2026 年 10 月

0 一句话本质(先记这个)

regmap 是内核的"寄存器访问抽象层"。你填一张只读的配置图纸 `regmap_config`,内核据此在内核堆上造出一个对你不透明的对象 `struct regmap`,并只给你一个指向它的指针(句柄)。之后所有读写都通过 `regmap_xxx(句柄, ...)` 函数间接进行——换 I2C/SPI 只改造对象时用的那一个入口函数,业务代码一行不动。

把两个最容易混的名字先钉死:

名字	是什么	谁创建	你能看到内部吗	定义在哪
<code>struct regmap_config</code>	图纸 / 配置单: 描述芯片寄存器特性(地址宽度、值宽度、最大地址.....)	你,写成 <code>const</code> 常量	能,字段全公开	<code>include/linux/regmap.h</code>
<code>struct regmap</code>	真正干活的对象: 内含总线读写回调、锁、缓存、从 config 拷来的参数	内核在 init 里 <code>kzalloc</code>	不能, 对驱动不透明	<code>drivers/base/regmap/internal.h</code> (藏起来)

类比: `regmap_config` 是**点菜单**(你勾"8 位地址、8 位数据、最大 0x75"); `devm_regmap_init_i2c()` 是**厨房**; `struct regmap` 是**做好的菜**(在厨房里),你只拿到一张**取餐号 = 指针**; `regmap_read()` 就是凭号找厨房要东西——你从不自己进厨房翻锅。

1 两个结构体:图纸 vs 机器

1.1 regmap_config:你交给内核的"订单"

```
static const struct regmap_config mpu6050_regmap_config = {
    .reg_bits = 8,          // 寄存器地址宽度(位)
    .val_bits = 8,          // 每个寄存器值的宽度(位)
    .max_register = 0x75, // 最大寄存器地址(WHO_AM_I)
};
```

这只是一张描述芯片特性的订单,本身不干活、不能读写寄存器。`const` 说明它是只读常量,编译进只读数据段。它的唯一作用:当原材料,喂给构造函数。

1.2 struct regmap:内核按订单造出的"机器"

```
struct regmap *map;    // 注意是指针!

map = devm_regmap_init_i2c(client, &mpu6050_regmap_config);
//          ↑客户端      ↑你的订单(图纸)
//          ↑返回一个指向"真正 regmap 对象"的指针
```

发生了什么:

```
你的 config(图纸)  |
                    |
client(走哪条总线) | -> devm_regmap_init_i2c() -> 内核 kcalloc 一个
                    | (工厂)                  struct regmap 对象
                    |
                    | 把 config 的字段拷进去、
                    | 把总线读写函数绑进去、
                    | 初始化锁、缓存.....
                    |
                    v
                    返回这个对象的地址 -> map
```

`map` 的值,就是内核堆上那个 `struct regmap` 对象的起始地址(一个指针)。你 `printk("%p", map)` 能打出来,但那个数字对你没意义——你只拿它当"身份证"用。

1.3 为什么内部要藏起来

public 头文件 `<linux/regmap.h>` 里只有一句前向声明 `struct regmap;`,不给你看字段。这叫**不透明指针** / **封装**,逼你只能通过函数操作:

```
regmap_read(map, 0x75, &val);    // ✓ 只能这样用
val = map->cache[0x75];          // ✗ 编译报错:incomplete type,看不到内部
```

好处:内核以后改 `struct regmap` 内部实现,你的驱动一行都不用改——因为你从没依赖它的内部长相。和 C++ 的 `private` 成员一个思想。

2 核心套路:第一个参数永远是句柄

几乎每个 regmap 函数第一个参数都是 `map` (或存起来的 `data->regmap`)。这不是巧合,是 C 语言的经典句柄(handle)模式:

```
regmap_read(map, reg, &val);
regmap_write(map, reg, val);
regmap_update_bits(map, reg, mask, val);
//          ↑ 全都是它打头
```

为什么? 这些函数自己不知道"该操作哪颗芯片、走哪条总线"。`map` 指向的对象里藏着总线读写函数、设备地址、锁、缓存等一切信息。你把 `map` 传进去,函数才知道:要读的是 I2C-1 总线上 0x68 那颗芯片,用 i2c 的读函数,读之前先抢 这个 `map` 的锁。

类比文件操作: `FILE *fp = fopen(...)` 得到句柄,之后 `fread(buf, 1, n, fp)`、`fclose(fp)` 都要带 `fp`,否则 `fread` 不知道读哪个文件。`map` = 操作对象的身份证,所以永远排第一个。

3 常用函数逐个拆:参数为什么是它

3.1 devm_regmap_init_i2c —— 造句柄(唯一第一参数不是 map)

```
struct regmap *devm_regmap_init_i2c(struct i2c_client *client,
                                    const struct regmap_config *config);
```

此刻 regmap 还没造出来,它正是来造 regmap 的,所以第一参数是 client。

参数	传什么	为什么要它
client	probe 收到的 client	告诉工厂:走哪条总线、芯片地址 0x68。工厂从 client 扒出 I2C 信息,绑定对应读写函数
config	&mpu6050_regmap_config	告诉工厂:寄存器 8 位、值 8 位、最大 0x75。传的是地址 & (指针),不是整个结构体拷贝

```
data->regmap = devm_regmap_init_i2c(client, &mpu6050_regmap_config);
if (IS_ERR(data->regmap)) // 失败返回"错误码伪装的指针"
    return PTR_ERR(data->regmap); // 用 IS_ERR/PTR_ERR 判断,不能用 == NULL
```

3.2 regmap_read —— 读一个寄存器

```
int regmap_read(struct regmap *map, unsigned int reg, unsigned int *val);

regmap_read(data->regmap, MPU6050_REG_WHO_AM_I, &whoami);
```

位置	参数	含义
第1	map	句柄:读哪颗芯片
第2	reg	寄存器地址:读这颗芯片的哪个寄存器
第3	val	结果往哪放:读到的值写进这个地址,所以要 &whoami

为什么第 3 个是 &whoami (取地址)? C 函数只能 return 一个值,而这里 return 已被"成功/失败错误码"占用。读到的数据只能用输出参数带出来:你给它一个变量地址,函数把数据填进去。

```
unsigned int whoami; // ① 先准备空变量
int ret = regmap_read(map, 0x75, &whoami); // ② 把它的地址给函数(内部执行 *val = 读到的值)
if (ret) // ③ ret==0 成功,非0是错误码(如 -EIO)
    return ret;
printf("WHO_AM_I = 0x%x\n", whoami); // 打印 0x68
```

两条铁律:① 函数的 return 值 = 错误码(0 成功,负数失败),不是数据;② 数据通过指针参数带出来,所以读要 &。

3.3 regmap_write —— 写一个寄存器

```
int regmap_write(struct regmap *map, unsigned int reg, unsigned int val);

regmap_write(data->regmap, MPU6050_REG_PWR_MGMT_1, 0x00);
```

第 3 个参数是**要写进去的值,直接给值,不是 &**。对比读/写最容易错的一点:

```
regmap_read (map, reg, &val); // 读:给地址(&),让函数往里塞    ← 数据流出芯片→进你变量
regmap_write(map, reg, val);  // 写:给值,函数拿去写出去        ← 数据从你这→进芯片
```

3.4 regmap_update_bits —— 只改某几位(最常用)

```
int regmap_update_bits(struct regmap *map, unsigned int reg,
                      unsigned int mask, unsigned int val);

// 唤醒 MPU6050:清除 PWR_MGMT_1(0x6B)的 bit6
regmap_update_bits(data->regmap, 0x6B, BIT(6), 0);
```

位置	参数	含义
第3	<code>mask</code>	哪些位要动(1=这位要改,0=保持不变)
第4	<code>val</code>	这些位改成什么

内部干的是读-改-写: $\text{新值} = (\text{旧值} \& \sim \text{mask}) \mid (\text{val} \& \text{mask})$ 。为什么需要 `mask`? 一个寄存器里塞了好几个功能位,你只想改 bit6(睡眠位),不能用 `regmap_write` 粗暴覆盖整字节——那会把其他位清掉。`update_bits` 帮你"只碰指定位",且**全程加锁**,不怕并发。

3.5 regmap_bulk_read —— 一次读一串连续寄存器

```
int regmap_bulk_read(struct regmap *map, unsigned int reg,
                    void *val, size_t val_count);

__be16 raw[7];
regmap_bulk_read(data->regmap, MPU6050_REG_ACCEL_XOUT_H, raw, sizeof(raw));
```

第2个是**起始**寄存器地址,第3个是缓冲区(数组名本身就是地址,不用 `&`),第4个是长度。MPU6050 的 X/Y/Z 加速度+温度+陀螺仪共 14 字节连续,一次读回,比调 14 次 `regmap_read` 快得多(只发一次总线事务)。

4 data->regmap:句柄存哪、为什么带 data->

probe 里造好的 `map` 是局部变量,probe 一返回就没了。但读数据、中断处理这些函数事后还要用它。
解决办法:把句柄存进驱动的私有数据结构。

```
/* ① 定义你自己的"随身包",把句柄装进去 */
struct mpu6050_data {
    struct regmap *regmap;    // ← 句柄存这儿
    struct device *dev;
    // ...其他要跨函数共享的东西
};

static int mpu6050_probe(struct i2c_client *client)
{
    struct mpu6050_data *data;

    data = devm_kzalloc(&client->dev, sizeof(*data), GFP_KERNEL);    // ② 分配随身包
    data->regmap = devm_regmap_init_i2c(client, &mpu6050_regmap_config);    // ③ 造句柄存进包
    i2c_set_clientdata(client, data);    // ④ 把包挂到 client 上

    unsigned int whoami;    // ⑤ 现在就能用
    regmap_read(data->regmap, MPU6050_REG_WHO_AM_I, &whoami);
    return 0;
}

/* 以后在读数据的函数里,取回随身包继续用 */
static int mpu6050_read_raw(/* ... */)
{
    struct mpu6050_data *data = /* 从 iio/client 取回那个随身包 */;
    __be16 raw[3];
    regmap_bulk_read(data->regmap, 0x3B, raw, sizeof(raw));
}
```

所以 `data->regmap` = "从我的随身包里取出之前存好的那个句柄"。写成 `data->regmap` 还是裸 `map`,**传进函数的东西完全一样**(都是那个指针),只是存放位置不同。

5 内存模型:在哪、能建几个、工厂是谁

5.1 对象在哪:内核空间,内核堆

铁定在**内核空间的堆**上。原因链:你的驱动跑在内核态; `regmap_init` 内部用 `kzalloc()` (内核版 `malloc`) 分配,给出的是**内核虚拟地址**;用户程序永远碰不到,要读传感器得通过 `/dev`、`/sys`、IIO 接口隔着一道墙,由内核在中间转一手。



地址长相类似 `0xfffff880_12340000` (内核态高地址),只在内核地址空间有意义。

5.2 建多个会怎样:各自独立,互不干扰

板上两颗 MPU6050(0x68 和 0x69),probe 被调两次,各造一个 regmap:

内核堆:

```
┌ struct regmap #1 ┐ 绑定 0x68、自己的锁、自己的缓存
└──────────────────┘
┌ struct regmap #2 ┐ 绑定 0x69、另一把锁、另一份缓存
└──────────────────┘
```

只有一份 `const config`(只读图纸,两次共用没关系—工厂只是照着抄)

- 两个对象是**两块不同内存**, `dataA->regmap != dataB->regmap` (两个不同指针)。
- 各带**独立的锁和缓存**:对 0x68 的操作不会阻塞 0x69。
- `const config` 图纸可被多颗同型号芯片**共享复用**。
- 靠不同指针区分操作对象: `regmap_read(dataA->regmap, ...)` 动 0x68, `regmap_read(dataB->regmap, ...)` 动 0x69。

5.3 "工厂"到底是谁

`devm_regmap_init_i2c` 只是**前台接待**,真正 `kzalloc` 造对象的在更深一层。完整调用链:

```
devm_regmap_init_i2c(client, config)  ← 你调用(宏,入口)
|
└─devm_regmap_init_i2c(...)           regmap-i2c.c
    | ① regmap_get_i2c_bus():挑一套"I2C 总线读写回调"(struct regmap_bus)
    ▼
    └─devm_regmap_init(dev, bus, ...)  登记"设备卸载时自动释放"
```

```

    ▼
__regmap_init(dev, bus, ..., config)  ★真正的工厂★  regmap.c
    | ② map = kzalloc(sizeof(*map), GFP_KERNEL); ← 对象在这里诞生
    | ③ 把 config 的 reg_bits/val_bits/max_register 拷进 map
    | ④ 把 bus 的读写回调绑进 map->reg_read / map->reg_write
    | ⑤ mutex_init(&map->mutex); 建缓存、建 debugfs...
    ▼
return map; ← 把新内存地址一路返回给你

```

层	文件	干什么
<code>devm_regmap_init_i2c</code> (前台)	regmap-i2c.c	入口,给你用的名字
<code>regmap_get_i2c_bus</code>	regmap-i2c.c	提供"I2C 专用"的总线读写函数(换 SPI 就是 regmap-spi.c 提供 SPI 版)
<code>__regmap_init</code>	regmap.c	真正的工厂:kzalloc 造对象、拷 config、绑回调、初始化锁和缓存

"最终工厂" = `drivers/base/regmap/regmap.c` 里的 `__regmap_init()`,总线无关的核心。

`devm_regmap_init_i2c/_spi/_mmio` 只是不同总线的入口,各自塞进"本总线怎么发时序"的回调,再汇流到 `__regmap_init`。这也是"换总线只改一个函数名"的原因:99% 的工作在共用的 `__regmap_init` 里。

6 max_register 到底什么意思

一句话:这颗芯片最大的、有效的寄存器地址编号。 MPU6050 内部寄存器地址从 0x00 排到 0x75(WHO_AM_I), 0x75 就是最后一个,所以 max_register = 0x75。这个值来自芯片数据手册的寄存器映射表,不是拍脑袋定的。

告诉 regmap 这个边界,它用来干三件事:

- 挡住越界访问:你不小心 regmap_read(map, 0x99, ...) (超过 0x75),regmap 可直接拦下返回错误,而不是去总线读一个不存在的寄存器拿回垃圾。
- 决定缓存 / debugfs 范围:缓存数组按 0x00~max_register 开大小; /sys/kernel/debug/regmap/.../registers 也只 dump 到 max_register。
- 配合 readable_reg/writeable_reg 回调界定合法区间。

填错的后果:

情况	后果
填小了(如 0x50)	0x50 以上的寄存器可能被判为非法,读写被拒,功能缺失
填大了(如 0xFF)	缓存/debugfs 多覆盖芯片其实没有的地址,dump 时读到无意义值或总线报错

类比:一本书有 118 页(0x00~0x75)。 max_register = 0x75 就是告诉图书管理员(regmap):"这本书最后一页是第 0x75 页,别让人翻到 0x76 页——那页不存在。"

7 速查总表

```
// 造句柄(第一参数是 client/config,因为还没有 regmap)
map = devm_regmap_init_i2c(client, &config);

// 之后全部以 map 句柄打头:
regmap_read      (map, reg, &val);           // 读→用 & 接收(数据是输出参数)
regmap_write     (map, reg, val);             // 写→直接给值
regmap_update_bits(map, reg, mask, val);      // 改指定位(读-改-写,带锁)
regmap_bulk_read (map, reg, buf, count);      // 连读多个连续寄存器
```

要点	记忆
第一个参数 map	身份证,告诉函数操作哪颗芯片;不给就不知道对谁下手
读 vs 写	读用 <code>&val</code> (给地址接数据),写用 <code>val</code> (直接给值);返回值一律是错误码,不是数据
config vs regmap	config 是公开的只读图纸;regmap 是内核堆上不透明的对象,你只有指针
data->regmap	只是把句柄存在私有结构里,方便 probe 之后的函数继续用
工厂	前台 devm_regmap_init_i2c → 核心 __regmap_init(真正 kzalloc)
max_register	最后一个有效寄存器地址,当合法上界;查数据手册

—— 完 ——